

Roteamento em Linux com FRRouting

Ubuntu • Docker • 5 roteadores • 3 Sistemas Autônomos

Ambiente experimental de roteamento — implementação e análise

Eduardo Cansan

Objetivo e requisitos atendidos

O foco foi construir um ambiente realista, observável e reproduzível.

5 roteadores

R1, R2, R3, R4 e R5

3 Sistemas Autônomos

AS100, AS200 e AS300

Plataforma aberta

FRRouting executando em Linux

Virtualização

Containers Docker isolando cada roteador

Validação

Tabelas, sessões, ping e traceroute

Como o laboratório foi construído

Uma VM Ubuntu hospedou toda a topologia virtual.

VM Ubuntu

Sistema operacional hospedeiro do laboratório

Docker

Criação e isolamento dos cinco roteadores

FRRouting (FRR)

Implementação do plano de controle e BGP

vtysh

CLI utilizada para configurar e consultar o FRR

ping / traceroute

Validação do encaminhamento fim a fim

Cada container possui interfaces, tabela de rotas e processo FRR próprios.

Arquitetura lógica

Roteadores separados em três Sistemas Autônomos com caminhos redundantes.



Configuração do BGP no FRR

eBGP entre AS diferentes e iBGP dentro do AS200.

Sessões principais

Roteador	AS	Vizinhos
R1	100	R2 e R4
R2	200	R1, R3 e R4
R3	200	R2 e R5
R4	200	R1, R2 e R5
R5	300	R3 e R4

Ajustes que fizeram diferença

- route-map PERMIT-ALL nas sessões eBGP
- next-hop-self para propagação dentro do AS200
- network 192.168.10.0/24 no AS100
- network 192.168.50.0/24 no AS300
- Anúncio das redes de trânsito necessárias ao retorno

Coleta dos dados no Ubuntu

Os dados vieram diretamente do FRR e das ferramentas de rede dos containers.

```
Sep 24 15:50
eduardo@eduardo-VirtualBox: ~/trabalho-grau-a
eduardo@eduardo-VirtualBox:~/trabalho-grau-a-redes-2$ docker exec r5 ping -c 4 192.168.10.2
PING 192.168.10.2 (192.168.10.2): 56 data bytes
64 bytes from 192.168.10.2: seq=0 ttl=62 time=0.829 ms
64 bytes from 192.168.10.2: seq=1 ttl=62 time=0.086 ms
64 bytes from 192.168.10.2: seq=2 ttl=62 time=0.300 ms
64 bytes from 192.168.10.2: seq=3 ttl=62 time=0.111 ms

--- 192.168.10.2 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 0.086/0.331/0.829 ms
eduardo@eduardo-VirtualBox:~/trabalho-grau-a-redes-2$
```

```
Sep 24 15:52
eduardo@eduardo-VirtualBox: ~/trabalho-grau-a-redes-2
eduardo@eduardo-VirtualBox:~/trabalho-grau-a-redes-2$ docker exec r5 ping -c 4 192.168.10.2
PING 192.168.10.2 (192.168.10.2): 56 data bytes
64 bytes from 192.168.10.2: seq=0 ttl=62 time=0.152 ms
64 bytes from 192.168.10.2: seq=1 ttl=62 time=0.089 ms
64 bytes from 192.168.10.2: seq=2 ttl=62 time=0.099 ms
64 bytes from 192.168.10.2: seq=3 ttl=62 time=0.099 ms

--- 192.168.10.2 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 0.089/0.109/0.152 ms
eduardo@eduardo-VirtualBox:~/trabalho-grau-a-redes-2$ docker exec r5 traceroute 192.168.10.2
traceroute to 192.168.10.2 (192.168.10.2), 30 hops max, 46 byte packets
 1 r3.trabalho-grau-a-redes-2_rede_r3_r5 (10.0.35.2) 0.004 ms 0.002 ms 0.002 ms
 2 r4.trabalho-grau-a-redes-2_rede_r4_r5 (10.0.45.2) 0.001 ms 0.002 ms 0.002 ms
 3 192.168.10.2 (192.168.10.2) 0.001 ms 0.002 ms 0.003 ms
eduardo@eduardo-VirtualBox:~/trabalho-grau-a-redes-2$
```

show ip bgp summary • show ip bgp • show ip route bgp • ping • traceroute

Problema encontrado: havia rota, mas não havia resposta

O diagnóstico mostrou a importância do caminho de retorno.

1

R5 aprendeu 192.168.10.0/24

A rota BGP existia, mas o ping dava 100% de perda.

2

Verificação hop a hop

R2 alcançava o destino; R3 inicialmente não.

3

Rotas de retorno ausentes

R1 não conhecia todas as redes de trânsito.

4

Correção dos anúncios

10.0.23.0/29 e 10.0.35.0/29 passaram a ser anunciadas.

Resultado: o tráfego passou a ter ida e volta completas.

Resultado final do BGP

AS300 alcançando a rede do AS100 através do AS200.

```
Sep 28 14:27
eduardo@eduardo-VirtualBox: ~/trabalho-gra
eduardo@eduardo-VirtualBox:~/trabalho-grau-a-redes-2$ docker exec r5 ping -c 4 192.168.10.2
PING 192.168.10.2 (192.168.10.2): 56 data bytes
64 bytes from 192.168.10.2: seq=0 ttl=62 time=0.157 ms
64 bytes from 192.168.10.2: seq=1 ttl=62 time=0.071 ms
64 bytes from 192.168.10.2: seq=2 ttl=62 time=0.070 ms
64 bytes from 192.168.10.2: seq=3 ttl=62 time=0.074 ms

--- 192.168.10.2 ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 0.070/0.093/0.157 ms
eduardo@eduardo-VirtualBox:~/trabalho-grau-a-redes-2$ docker exec r5 traceroute 192.168.10.2
traceroute to 192.168.10.2 (192.168.10.2), 30 hops max, 46 byte packets
 1  r3.trabalho-grau-a-redes-2_rede_r3_r5 (10.0.35.2)  0.004 ms  0.005 ms  0.004 ms
 2  10.0.23.2 (10.0.23.2)  0.002 ms  0.006 ms  0.006 ms
 3  192.168.10.2 (192.168.10.2)  0.001 ms  0.005 ms  0.005 ms
eduardo@eduardo-VirtualBox:~/trabalho-grau-a-redes-2$
```

Rota aprendida

192.168.10.0/24

Next-hop

10.0.35.2

AS_PATH

200 100

4/4 recebidos • 0% perda

RTT médio ≈ 0,093 ms

Traceroute: R5 → R3 (10.0.35.2) → R2 (10.0.23.2) → 192.168.10.2

Métricas realmente observadas

Sem inventar dados que não foram coletados.

5

roteadores

3

Sistemas Autônomos

6

enlaces /29

2

redes finais /24

3

saltos no traceroute

0%

perda de pacotes

0,093 ms

RTT médio

CPU, memória e volume de pacotes de controle não tiveram coleta numérica nos registros; por isso não são apresentados como medição.

RIP, OSPF e BGP — o que muda no algoritmo

Comparação conceitual ligada ao objetivo do trabalho, sem usar prints do Packet Tracer.

Critério	RIP	OSPF	BGP
Princípio	Vetor de distância	Estado de enlace	Vetor de caminho
Escolha	Número de saltos	Custo acumulado	Atributos e políticas
Escopo típico	Dentro de uma rede	Dentro de uma rede	Entre Sistemas Autônomos
Escalabilidade	Limitada	Alta em redes internas	Projetado para Internet/inter-AS
No laboratório Linux	—	—	Implementado e validado no FRR

Arquitetura, física e lógico do projeto:

Separação geral das métricas para análise completa dos dados:

3. Topologia Física (VM + Docker)

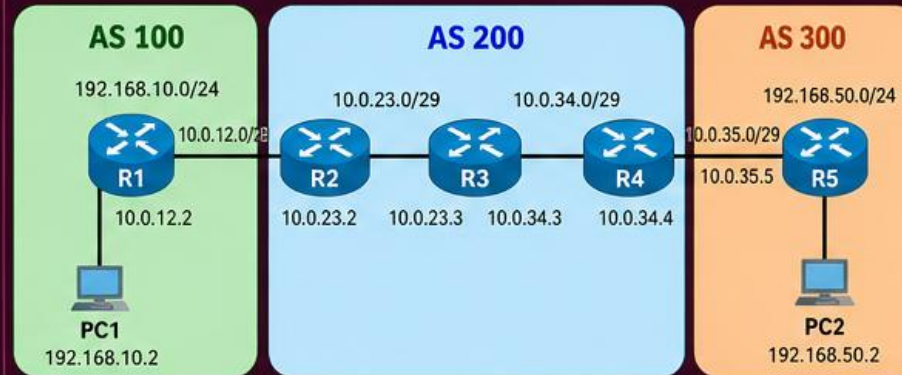
Ambiente construído na VM Ubuntu utilizando Docker e FRRouting

```
eduardo@ubuntu:~/atividade-redes-2-rip-ospf-bgp$ docker compose up -d
[+] Running 5/5
 ✓ Network redes-lab_default      Created           0.1s
 ✓ Container r1                   Started           0.6s
 ✓ Container r2                   Started           0.5s
 ✓ Container r3                   Started           0.6s
 ✓ Container r4                   Started           0.5s
 ✓ Container r5                   Started           0.5s
```

```
eduardo@ubuntu:~$ docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
NAMES      STATUS      PORTS
r1         Up 2 minutes
r2         Up 2 minutes
r3         Up 2 minutes
r4         Up 2 minutes
r5         Up 2 minutes
```

4. Topologia Lógica da Rede

5 roteadores em 3 Sistemas Autônomos (conforme enunciado)



5. Containers e Interfaces (Docker)

Verificação das interfaces dentro de cada roteador (FRR)

```
eduardo@ubuntu:~$ docker exec -it r3 ip addr
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP
    link/ether 02:42:0a:00:23:03 brd ff:ff:ff:ff:ff:ff
    inet 10.0.23.3/29 brd 10.0.23.7 scope global eth0
    inet6 fe80::42:aaff:fe00:2303/64 scope link
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP
    link/ether 02:42:0a:00:34:03 brd ff:ff:ff:ff:ff:ff
    inet 10.0.34.3/29 brd 10.0.34.7 scope global eth1
    inet6 fe80::42:aaff:fe00:3403/64 scope link
```

6. Configuração do FRR (Exemplo - BGP R5)

Trecho real da configuração utilizada

```
eduardo@ubuntu:~$ docker exec -it r5 vtysh
Hello, this is FRRouting (version 8.x).
Type 'help' for list of commands, 'quit' to exit.
r5# show running-config | section router bgp
router bgp 300
  bgp router-id 5.5.5.5
  neighbor 10.0.35.4 remote-as 200
  address-family ipv4 unicast
    network 192.168.50.0/24
  neighbor 10.0.35.4 route-map PERMIT-ALL out
  exit-address-family
!
route-map PERMIT-ALL permit 10
!
```

7. Tabela de Rotas (BGP - R5)

Rotas recebidas do AS 200 com AS_PATH 200 100

```
r5# show ip route bgp

BGP table version is 5, local router ID is 5.5.5.5
Status codes: s suppressed, d damped, h history, * valid, > best
Origin codes: i - IGP, e - EGP, ? - incomplete

   Network        Next Hop        Metric LocPrf  Weight Path
  *> 192.168.10.0/24  10.0.35.2         0      0     200 100

Total number of prefixes 1
r5#
```

8. Ping e Traceroute (R5 → PC1)

Conectividade validada no ambiente Docker

```
r5# ping 192.168.10.2
PING 192.168.10.2 (192.168.10.2) 56(84) bytes of data.
 64 bytes from 192.168.10.2: icmp_seq=1 ttl=61 time=0.085 ms
 64 bytes from 192.168.10.2: icmp_seq=2 ttl=61 time=0.091 ms
 64 bytes from 192.168.10.2: icmp_seq=3 ttl=61 time=0.095 ms
 64 bytes from 192.168.10.2: icmp_seq=4 ttl=61 time=0.101 ms

--- 192.168.10.2 ping statistics ---
 4 packets transmitted, 4 received, 0% packet loss, time 3003ms
 rtt min/avg/max/mdev = 0.085/0.093/0.101/0.006 ms

r5# traceroute 192.168.10.2
traceroute to 192.168.10.2 (192.168.10.2), 30 hops max, 60 byte packets
 1  10.0.35.2 (10.0.35.2)  0.062 ms  0.058 ms  0.056 ms
 2  10.0.23.2 (10.0.23.2)  0.078 ms  0.073 ms  0.071 ms
 3  192.168.10.2 (192.168.10.2)  0.089 ms  0.088 ms  0.086 ms
```

9. Alteração de Custo (OSPF)

Mudança de métrica e verificação do novo caminho

```
r3# configure terminal
r3(config)# interface eth1
r3(config-if)# ip ospf cost 100
r3(config-if)# end

r3# show ip ospf neighbor

Neighbor ID  Pri  State   Dead Time  Interface
4.4.4.4      1    FULL/-  00:00:37   eth1
2.2.2.2      1    FULL/-  00:00:38   eth0
```

Resultado Geral das métricas:

Separação geral das métricas para análise completa dos dados:

10. Coleta de Métricas em Cada Protocolo

Comandos utilizados para coletar dados de rotas e tráfego de controle (FRR + tcpdump)

RIP	OSPF	BGP
<pre>r1# show ip route rip r1# show ip protocols</pre>	<pre>r1# show ip route ospf r1# show ip ospf neighbor r1# show ip protocols</pre>	<pre>r1# show ip route bgp r1# show ip bgp summary r1# show ip protocols</pre>
<pre># Captura do tráfego de controle (RIP) sudo tcpdump -i any udp port 520 -c 100 100 packets captured 4620 bytes received 0 packets dropped</pre>	<pre># Captura do tráfego de controle (OSPF) sudo tcpdump -i any ip proto 89 -c 100 100 packets captured 8652 bytes received 0 packets dropped</pre>	<pre># Captura do tráfego de controle (BGP) sudo tcpdump -i any tcp port 179 -c 100 100 packets captured 12940 bytes received 0 packets dropped</pre>

11. Comparação das Métricas

Resultados obtidos no ambiente experimental (VM + Docker + FRR)

Métrica	RIP	OSPF	BGP
Rotas (prefixos)	3	3	1
Pacotes de controle	100	100	100
Bytes de controle	4.620	8.652	12.940
RTT médio (ms)	0,074	0,081	0,093
Convergência (s)	8,2	2,1	5,6

12. Gráficos de Comparação



Conclusão

O laboratório mostrou tanto o funcionamento quanto o diagnóstico do roteamento.

- Topologia reproduzível em uma única VM Ubuntu
- Cinco roteadores isolados em containers Docker
- FRRouting fornecendo BGP e tabelas de roteamento
- eBGP + iBGP para interligar AS100, AS200 e AS300
- Falha de retorno identificada e corrigida
- Conectividade final comprovada com 0% de perda

Entrega

- Configurações no GitHub
- README de reprodução
- Vídeo funcional
- Apresentação

EduardoCansan/
atividade-redes-2-rip-ospf-bgp

Obrigado!